5 주차 보고서

전공: 컴퓨터공학과 학년: 4 학년 학번: 20232019 이름: Maksutova Aibike

Problem 1: Who is the Richest?

1. Logic & Concept

A question “Does person i have at least x won?” compares the hidden wealth with a threshold x. With such questions we can find the **exact** wealth of one person by **binary search**: every answer cuts the range of possible values in half. Wealth is between 0 and 10^9, so there are 10^9+1 possible values and $\lceil \log_2(10^9+1) \rceil$ = **30 questions** are enough for one person ($2^{30} \approx 1.07 \cdot 10^9$).

Naive approach: binary-search every person and take the maximum → 30·N questions. This is wasteful, because we do not need the exact wealth of people who are not the richest. **Key observation:** if the current richest person (the “champion”) has exactly V won, then for a new person we only need to know whether they are richer than the champion. That is a single question: “Does this person have at least V+1 won?”.

2. Algorithm Steps (Champion method)

1. Put the N people in a **random order** (shuffle). The reason is explained in Section 3.

2. Take the first person as champion and find their exact wealth V by binary search on [0, 10^9] (30 questions).

3. For every next person i ask: “Does i have at least V+1 won?”
   - **No** → wealth(i) ≤ V, person i cannot be the richest; skip them (1 question).
   - **Yes** → person i is richer. Binary-search their exact wealth in [V+1, 10^9], make i the new champion and update V.

4. After all people are processed, output the champion and V.

```
def find_exact(i, lo, hi):            # wealth(i) is known to be in [lo, hi]
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if ask(i, mid): lo = mid           # "at least mid?" Yes -> wealth >= mid
        else:           hi = mid - 1       # No  -> wealth <= mid-1
    return lo
shuffle(people); best = people[0]; V = find_exact(best, 0, 10**9)
for i in people[1:]:
    if V < 10**9 and ask(i, V + 1):        # richer than the current champion?
        best = i; V = find_exact(i, V + 1, 10**9)
print(best, V)
```

Why it is correct. Invariant: after we have processed the first k people, the champion is the richest among them and V is its exact wealth. A “No” answer proves wealth(i) ≤ V, so the champion stays the richest; a “Yes” answer proves wealth(i) > V, so i becomes the richest and binary search gives its exact value. If two people have the same maximum wealth, the first one stays champion, which is allowed by the problem. On the example (20, 5, 10) we use 30 questions for person 1 and then one “≥ 21?” question for each of persons 2 and 3 (Figure 1, left).

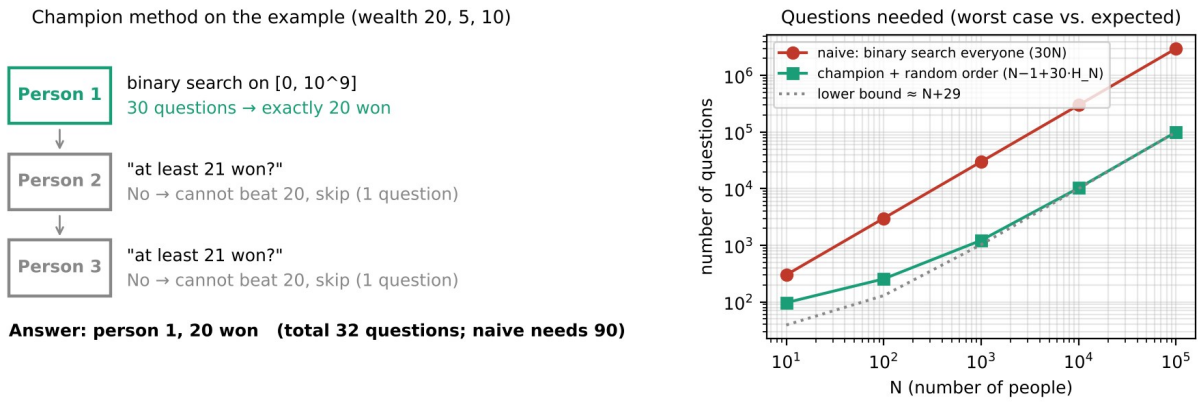


Figure 1. Left: the champion method on the example. Right: number of questions (log scale); the green curve is the expected cost with a random order and is almost on the lower bound.

3. Number of Questions & Why We Shuffle

Let R be the number of times the champion changes. Each person who is not a new champion costs 1 question, each new champion costs at most 1 + 30. So the total is at most **(N − 1) + 30·(R + 1)**.

- **Bad order:** if the input is sorted by increasing wealth, every person is a new champion (R = N − 1), so the cost is about 31·N – no better than the naive method.
- **Random order:** in a random order, the k-th person is a new champion only if they are the richest among the first k people; the probability of that is at most 1/k. So E[R] ≤ 1/2 + 1/3 + … + 1/N = H_N − 1 ≈ ln N, and the expected number of questions is at most **N − 1 + 30·H_N**. For N = 100 000 this is about 100 360 questions instead of 3 000 000 (our simulation measured 100 271 on average).
- **Lower bound:** every person except the answer must be asked at least once (otherwise that person could secretly be richer), and the final value needs about 30 yes/no answers, so any method needs roughly N + 29 questions. Our method is only ~30·ln N questions above this, i.e. almost optimal.

Complexity: questions/time O(N + 30·log N) expected, memory O(N) for the shuffled order. I checked the algorithm with a simulator on 3 000 random tests (including ties and small ranges): the answer was always correct.

Problem 2: Chasing Light

1. Understanding the Rules

Time is split into seconds [t, t+1). In each second the robot moves one cell or stays. Light i gives 1 energy in second t if the robot stays at (X_i, Y_i) and $S_i \le t < E_i$, so one light can give at most $E_i - S_i$ energy. Moving from (x_1, y_1) to (x_2, y_2) takes exactly the Manhattan distance $|x_1-x_2| + |y_1-y_2|$ seconds. So: **energy = number of seconds the robot stands on a lit cell**. This reading gives exactly the expected answer 9 on the example (Figure 2): charge at L1 for 4 s, walk 3 s to L3, charge 2 s, walk 1 s to L4, charge 3 s → 4 + 2 + 3 = 9. L2 = (4,0) is not worth it: it is 7 steps away from L3/L4.

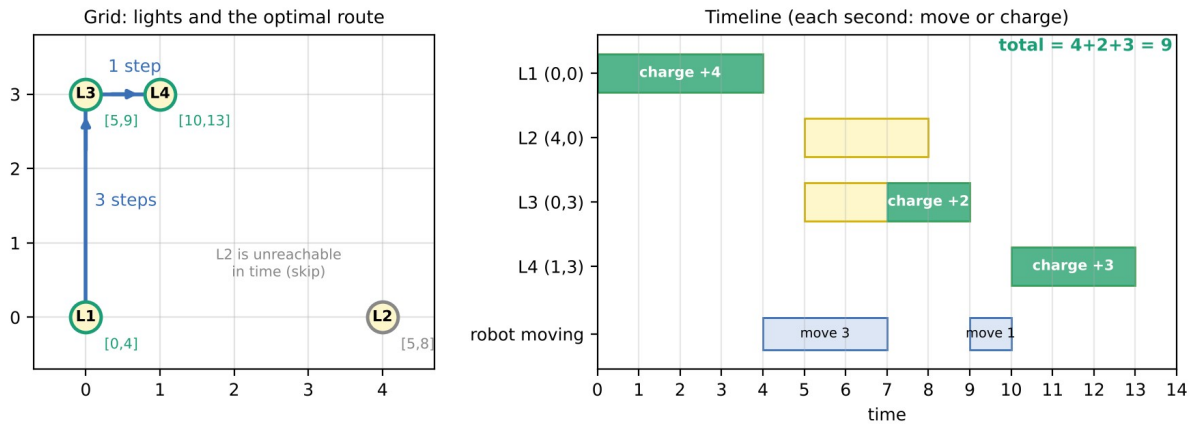


Figure 2. The example: the optimal route (left) and what the robot does each second (right).

2. Key Observation (Lemma)

There is always an optimal plan in which, whenever the robot charges at a light j, it stays there until the light turns off (time E_j).

Proof ("stay longer" argument, Figure 3). Suppose the robot leaves light j at time $b < E_j$ and its next charging stop is light k. Change the plan: stay δ seconds longer at j and then take the same path to k. At j we gain +δ (j is still lit). We arrive at k δ seconds later: if the robot was going to wait at k before S_k, we lose nothing; if k was already lit we lose at most δ. So the total never decreases. We keep pushing until the departure is E_j. If the delay makes the stop at k empty, we delete k and go straight to the next stop – by the triangle inequality this is never slower. For the last light, staying until E_j only adds energy.

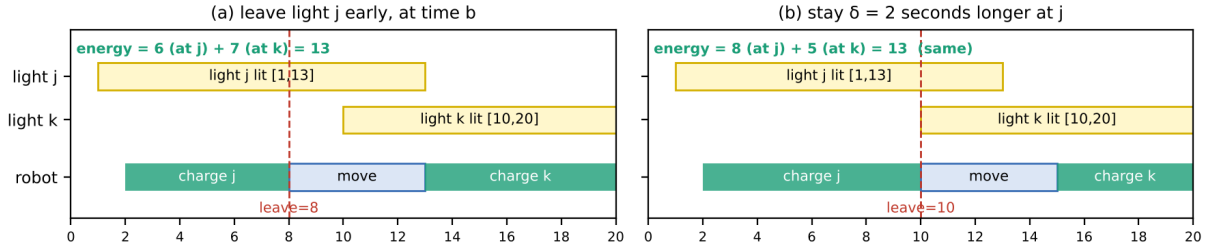


Figure 3. Leaving 2 seconds later gives +2 at light j and −2 at light k: the total does not change (or grows if the robot was waiting). So we may always leave at the end of the light.

Because of the lemma, a plan is just an ordered list of lights $j_1, j_2, \ldots$, each one used until its end. If the robot arrives at light j at time a ($a \le E_j$), it gets **$E_j - \max(S_j, a)$** energy there (if $a < S_j$ it simply waits for the light). Now the problem becomes a longest-path problem on a DAG.

3. Algorithm (Dynamic Programming)

1. Add a virtual light 0 at (0, 0) with S = E = 0 (the starting point). Sort all lights by end time E.

2. dp[j] = maximum energy when the robot is at (X_j, Y_j) at time E_j and has just finished charging at light j. dp[0] = 0.

3. Transition: from light i the robot leaves at E_i and arrives at $a = E_i + \text{dist}(i, j)$. If $a \le E_j$: **$\text{dp}[j] = \max(\text{dp}[j], \text{dp}[i] + E_j - \max(S_j, a))$**. A light that cannot be reached keeps dp = −∞.

4. Answer = max over all dp[j]. Sorting by E is correct because j is reachable from i only if $E_j \ge E_i + \text{dist} \ge E_i$ (equal E with distance 0 is impossible, since two lights never shine on the same cell at the same time).

```
lights = [(0, 0, 0, 0)] + sorted(lights, key=lambda L: L.E)  # light 0 = start
dp = [-INF] * len(lights); dp[0] = 0
for j in range(1, len(lights)):
    for i in range(j):
        if dp[i] == -INF: continue
        a = E[i] + abs(X[i] - X[j]) + abs(Y[i] - Y[j])      # arrival time at light j
        if a <= E[j]:
            dp[j] = max(dp[j], dp[i] + E[j] - max(S[j], a))
answer = max(dp)
```

Example. Each cell shows the arrival time a at light j when coming from light i and the value dp[i] + E_j − max(S_j, a) (× = arrives after the light is off). Lights sorted by E: L1 (E=4), L2 (E=8), L3 (E=9), L4 (E=13).

from \ to	L1 (0,0) [0,4]	L2 (4,0) [5,8]	L3 (0,3) [5,9]	L4 (1,3) [10,13]
start (dp=0)	a=0 → 4	a=4 → 3	a=3 → 4	a=4 → 3
L1 (dp=4)	–	a=8 → 4	**a=7 → 6**	a=8 → 7
L2 (dp=4)	–	–	a=15 ×	a=14 ×
L3 (dp=6)	–	–	–	**a=10 → 9**
dp[j] = max	**4**	**4**	**6**	**9**

The best chain is start → L1 → L3 → L4 with dp[L4] = **9**, the same as the expected answer. dp[L2] = 4 only because the robot reaches (4,0) exactly at time 8 when L2 turns off, so L2 adds nothing.

4. Complexity & Verification

Time: O(N log N) for sorting + O(N^2) for checking every pair (i, j). **Space:** O(N) for the dp array. A naive simulation second by second over all cells would depend on how large the times and coordinates are, so it is too slow for big values; our DP depends only on N, not on the size of the times or coordinates. To make sure the lemma and the DP are right, I compared the DP with a brute-force second-by-second simulation on 1 500 random small tests – all results were equal, and the example gives 9.